PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

IntelliShell: A Smart, Resource-Aware, and Adaptive Command-Line Interface

Student / Team Information

Team Name:	*Thread Masters*
Vibhi Rawat (Team Lead) *University Roll no: 2319847* *Student ID: 23012642* *Email: rawatvibhi2020@gmail.com*	
Aaradhya Chachra *University Roll no: 2318112* *Student ID: 23011499* *Email: aaradhyachachra779@gmail.com*	
Rohit Das *University Roll no: 2319847* *Student ID: 23012642* *Email: harsh.7das@gmail.com*	

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Traditional command-line shells like Bash or Zsh are powerful, but they often lack features to simplify usage for beginners and to intelligently adapt to system context. Users may struggle with remembering commands, managing resources, or performing repetitive tasks efficiently.

Our motivation is to design a **smart and resource-aware shell** that not only executes commands but also **assists, learns, and adapts**. The shell will provide **real-time command suggestions**, **auto-aliasing**, **shortcuts for file and network management**, and even **role-based access control** to ensure security.

This project is important because it merges **usability and intelligence** in operating systems. It will benefit both novice users (through suggestions and shortcuts) and advanced users (through automation and efficiency)

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Today, shells like Bash, Zsh, Fish are widely used. Fish shell, for instance, provides some command suggestions, while Bash offers scripting flexibility. However:

- Command suggestions are not context-aware in most shells.
- Resource-awareness (e.g., warning when a process may overload CPU/RAM) is missing.
- Built-in shortcuts for file/network tasks are minimal.
- Learning/adaptation (auto-alias) is mostly manual.

Our solution builds upon these but integrates them into one cohesive shell.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

Goals:

- Build a smart shell that improves usability and efficiency.
- Integrate AI-powered suggestions and resource-awareness.
- Provide shortcuts for common file/network tasks.
- Support multi-session handling and role-based access control (optional advanced features).

Milestones:

Week 1–2: Literature review, finalize architecture & technology stack.
Week 3–4: Implement core shell functionalities (command parsing, execution).
Week 5–6: Add smart command suggestions & auto-aliasing.
Week 7: Implement file/network management shortcuts.
Week 8: Add resource-awareness features.
Week 9: Multi-session handling & RBAC (if time permits).
Week 10: Testing, debugging, and documentation.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

We will design the shell in C/C++ with system calls for process management, I/O handling, and command execution. For resource monitoring, we will integrate Linux APIs like /proc filesystem.

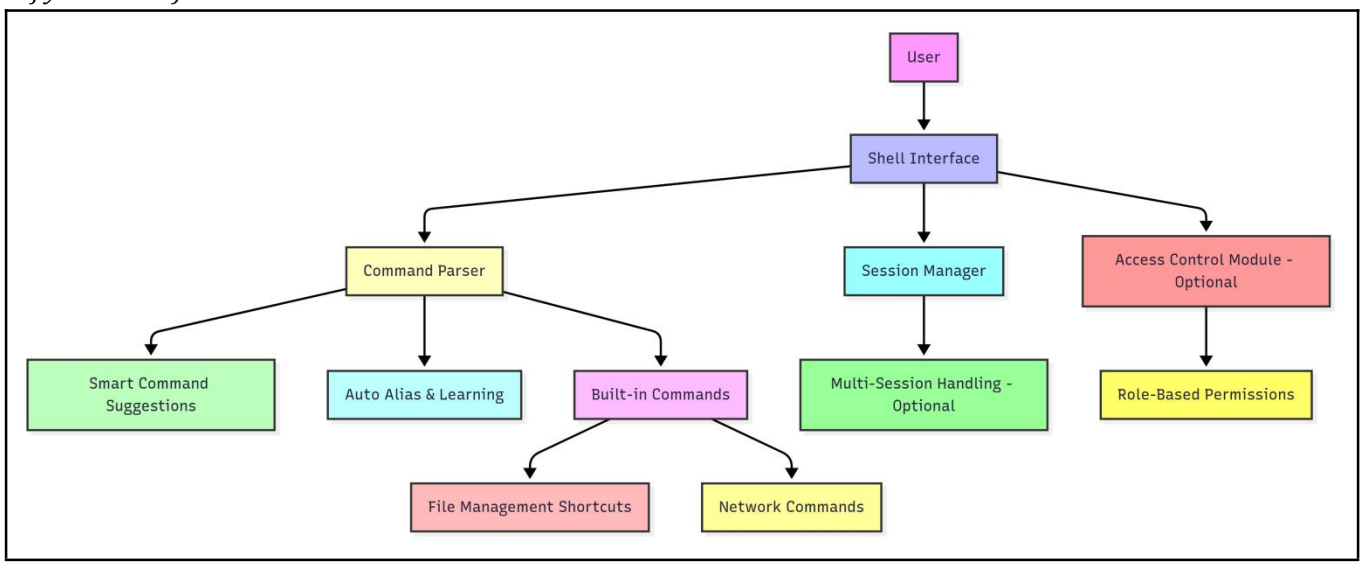
Key technologies & tools:

Language: C/C++
OS: Linux (Ubuntu environment)
Libraries: ncurses (for UI), POSIX APIs (fork, exec, pipes), system monitoring APIs.
Version Control: GitHub
Documentation & Reporting: LaTeX/Markdown

The shell will follow a modular architecture:
Command Parser – interpret user input.
Execution Engine – handle processes & I/O.
Suggestion Engine – AI-like recommendations & aliasing.
Resource Monitor – detect CPU/RAM usage.
File/Network Module – shortcuts for file operations & network commands.
Security Layer (RBAC) – optional role-based restrictions.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

At the end of this project, we will deliver:

- A working smart shell prototype with implemented features.
- Source code (GitHub).
- User manual and documentation.
- Performance comparison with traditional shells.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

- Users will run the shell in a Linux environment.
- Core Linux system calls/APIs will be available.
- AI suggestion module will be rule-based (lightweight) rather than heavy ML models due to time constraints.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

- Advanced Programming in the UNIX Environment – W. Richard Stevens.
- Linux Manual Pages (man bash, man exec, man fork).
- Fish Shell Documentation: https://fishshell.com/
- POSIX Standards: https://pubs.opengroup.org/onlinepubs/9699919799/